



PROGRAMMATION FONCTIONNELLE via le langage Haskell

A. OUALI

L2 au département Mathématique-Informatique
UPR des sciences
Université de Caen Normandie



Fonctions d'ordre supérieur : concept de base

- Encapsuler des modèles de programmation communs en tant que fonctions grâce à la Curryfication.

```
add :: Int -> Int -> Int
add x y = x + y
-- s'écrit
add :: Int -> (Int -> Int)
add = \x -> (\y -> x + y)
```

- Une fonction qui prend une fonction et une valeur, et renvoie le résultat d'appliquer la fonction deux fois à la valeur.

```
deuxFoisF :: (a -> a) -> a -> a
deuxFoisF f x = f (f x)
```

- Application :

```
Prelude> deuxFoisF (*2) 3
12
```

A. Ouali

Programmation fonctionnelle en Haskell

2 / 38

Fonctions d'ordre supérieur : map

Multiple applications d'une fonction

- `(map f xs)` construit la liste des applications de la fonction `f` à tous les éléments de la liste `xs`

```
> map square [1..4] ==> [1,4,9,16]
> map odd [1..4] ==> [True, False, True, False]
> map (* 2) [1..4] ==> [2, 4, 6, 8]
> map even [1..4] ==> [False, True, False, True]
```

- typage

```
map :: (a -> b) -> [a] -> [b]
```

- somme des carrés des `n` premiers entiers

```
sumSquare :: Int -> Int
sumSquare n = sum (map square [1..n])
```

A. Ouali

Programmation fonctionnelle en Haskell

3 / 38

Fonctions d'ordre supérieur

Multiple applications d'une fonction

- définition récursive de la fonction `map`

```
map :: (a -> b) -> [a] -> [b]
map f [] = []
map f (x:xs) = (f x) : (map f xs)
```

- autre définition à l'aide d'une ZF-expression

```
map f xs = [f x | x <- xs]
```

A. Ouali

Programmation fonctionnelle en Haskell

4 / 38

Fonctions d'ordre supérieur

Multiple applications d'une fonction

- distributivité de `map` par rapport à la composition de fonctions

```
map (f.g) = (map f) . (map g)
```

- distributivité de `(map f)` par rapport à la concaténation

```
map f (xs ++ ys) = (map f xs) ++ (map f ys)
```

A. Ouali

Programmation fonctionnelle en Haskell

5 / 38

Fonctions d'ordre supérieur : filter

Filtrer les éléments d'une liste satisfaisant une propriété

- `(filter p xs)` construit la liste des éléments de la liste `xs` satisfaisant le prédicat `p`

```
> filter even [1..10] ==> [2,4,6,8,10]
> filter odd [1..10] ==> [1,3,5,7,9]
> filter (== 'a') "azertyaap" ==> "aaaa"
> filter (/= 'a') "azertyaap" ==> "zertyp"
```

- typage

```
filter :: (a -> Bool) -> [a] -> [a]
```

- somme des carrés des nombres pairs entre 1 et `n`

```
sumEvenSquare :: Int -> Int
sumEvenSquare n = sum (map square (filter even [1..n]))
```

A. Ouali

Programmation fonctionnelle en Haskell

6 / 38

Fonctions d'ordre supérieur : filter

Filtrer les éléments d'une liste satisfaisant une propriété

- définition récursive de `filter`

```
filter :: (a -> Bool) -> [a] -> [a]
filter p [] = []
filter p (x:xs) =
  if (p x)
    then (x : (filter p xs))
    otherwise = filter p xs
```

- autre définition à l'aide d'une ZF-expression

```
filter p xs = [x | x <- xs, p x]
```

A. Ouali

Programmation fonctionnelle en Haskell

7 / 38

Fonctions d'ordre supérieur : filter

Filtrer les éléments d'une liste satisfaisant une propriété

- commutativité par rapport à la composition de fonctions

```
(filter p) . (filter q) = (filter q) . (filter p)
```

- distributivité de `(filter p)` sur `(++)`

```
filter p (xs ++ ys) = (filter p xs) ++ (filter p ys)
```

A. Ouali

Programmation fonctionnelle en Haskell

8 / 38

Fonctions d'ordre supérieur : all , any, takeWhile, dropWhile



- Vérifier si tous les éléments d'une liste satisfont un prédicat :

```
> all even [2, 4, 6, 8]
True
```

- Vérifier si un élément d'une liste satisfait un prédicat :

```
> any odd [2, 4, 6, 8]
False
```

- Retourner les éléments dans une liste si le prédicat est satisfait :

```
Prelude> takeWhile (/=' ') "abc def"
"abc"
```

- Retourner les éléments dans une liste en Enlevant les éléments satisfaisant le prédicat :

```
Main> dropWhile (/=' ') "abc def"
" def"
```

A. Ouali

Programmation fonctionnelle en Haskell

9 / 38

Fonctions d'ordre supérieur : fonctions anonymes



Retour sur les fonctions anonymes (λ-abstraction)

- Les fonctions anonymes peuvent utiliser le filtrage (suite)

```
> let xs = [("Jean",23),("Marie",25),("Paul",30),("Anne",18)]

-- les moins de 25 ans
> filter \(l, age) -> (age <= 25) xs
[("Jean",23),("Marie",25),("Anne",18)]

-- Les couples d'entiers ordonnés
> filter \(x,y) -> (x<y) [(1,4), (5,2), (2, 9)]
[(1,4), (2,9)]

lesordones :: [(Int, Int)] -> [(Int, Int)]
lesordones xs = filter \(x,y) -> (x<y) xs
```

A. Ouali

Programmation fonctionnelle en Haskell

11 / 38

Fonctions d'ordre supérieur : foldr



Fold (à droite)

- généraliser à une liste l'application d'un opérateur binaire
- foldr associe à droite i.e. l'application commence par la fin de liste

```
foldr op e [x1, x2, ..., xn] =
  x1 op (x2 op (x3 ... op (xn op e) ...))

foldr (+) 0 [2,5,7,9] = 2 + (5 + (7 + (9 + 0))) = 23
foldr (+) 1 [2,5,7,9] = 2 + (5 + (7 + (9 + 1))) = 630
```

- fonctions usuelles exprimées à l'aide de foldr

```
sum      = foldr (+) 0
product = foldr (*) 1
and      = foldr (&&) True
or       = foldr (||) False
```

A. Ouali

Programmation fonctionnelle en Haskell

13 / 38

Fonctions d'ordre supérieur

Fold (à droite)

- foldr associe à droite : l'application commence par la fin de liste

```
foldr op e [] = e
foldr op e [x1,...,xn] = x1 op (x2 ... op (xn op e) ...)
```

- typage (exemples étudiés jusque là)

```
(op) :: (a -> a -> a)
foldr :: (a -> a -> a) -> a -> [a] -> a
```

- typage (cas général)

```
(op) :: (a -> b -> b)
foldr :: (a -> b -> b) -> b -> [a] -> b
```

A. Ouali

Programmation fonctionnelle en Haskell

15 / 38

Fonctions d'ordre supérieur : fonctions anonymes



Retour sur les fonctions anonymes (λ-abstraction)

- Eviter de définir des fonctions de peu d'intérêt général ou bien utilisées ponctuellement

```
Prelude> map (\x->x+1) [1..10]
[2,3,4,5,6,7,8,9,10,11]

Prelude> filter (\x->(5<x && x<10)) [1..20]
[6,7,8,9]
```

- Les fonctions anonymes peuvent utiliser le filtrage

```
> let xs = [("Jean",23),("Marie",25),("Paul",30),("Anne",18)]

-- les moins de 25 ans
> filter \(nom,age) -> (age <= 25) xs
[("Jean",23),("Marie",25),("Anne",18)]
```

A. Ouali

Programmation fonctionnelle en Haskell

10 / 38

Récursivité : Base et Héritéité



- Fonction somme récursive utilisant les gardes :

```
1 somme x
2
3 | x == 0 = 0 -- case de base
4
5 | x > 0 = x + somme (x - 1) -- cas récursif (Hérédité)
6
7 -- Autres fonctions utilisant la récursivité
8 product [] = 1
9 product (x : xs) = x * product xs
10
11 or [] = False
12 or (x : xs) = x || (or xs)
13
14 and [] = True
15 and (x : xs) = x && (and xs)
```

A. Ouali

Programmation fonctionnelle en Haskell

12 / 38

Fonctions d'ordre supérieur

La fonction concat généralise l'opérateur binaire (++)

```
> concat [[1,2],[2,3],[3,4]] ==> [1,2,2,3,3,4]
> [1,2] ++ [2,3] ++ [3,4] ==> [1,2,2,3,3,4]

> concat ["nous ", "formons ", "une ", "liste"]
==> "nous formons une liste"
```

- typage

```
concat :: [[a]] -> [a]
```

- concat définie par un foldr

```
concat xss = foldr (++) [] xss
```

- concat définie par une ZF expression

```
concat xss = [x | xs <- xss, x <- xs]
```

A. Ouali

Programmation fonctionnelle en Haskell

14 / 38

Fonctions d'ordre supérieur



Fold (à droite)

- dans les exemples considérés, les opérateurs sont de type (a -> a -> a), associatifs et possédant un neutre e. Le couple (op, e) forme un monoïde¹
- si (op, e) forme un monoïde, alors

```
foldr op e [] = e
foldr op e [x1,x2,...,xn] = x1 op x2 ... op xn
```

- mais tous les couples (op, e) ne forment pas un monoïde

```
length :: [a] -> Int
length = foldr op 0
  where op x n = n + 1
```

1. op est une loi de composition associative et e un élément neutre.

A. Ouali

Programmation fonctionnelle en Haskell

16 / 38



Fold (à gauche)

► foldl associe à gauche : on commence l'application par le début de liste

```
foldl op e [] = e
foldl op e [x1,...,xn] = (...((e op x1) op x2) ... op xn)
```

► exemples :

```
reverse :: [a] -> [a]
reverse = foldl op []
  where op xs x = x:xs

pack :: [Int] -> Int
pack = foldl op 0
  where op n x = 10*n + x
```



Fold (propriétés)

(P₄) deux autres propriétés par rapport à l'opérateur (++)

```
foldl op e (xs ++ ys) = foldl op (foldl op e xs) ys
foldr op e (xs ++ ys) = foldr op (foldr op e ys) xs
```



Fold (propriétés)

(P₁) si (op,e) forme un monoïde, alors : (foldr op e) = (foldl op e)

(P₂) soient op1 et op2 tq
 (i) x op1 (y op2 z) = (x op1 y) op2 z
 (ii) x op1 e = e op2 x
alors (foldr op1 e) = (foldl op2 e)

(P₃) soient op et e, alors il existe op' tq

```
foldr op e xs = foldl op' e (reverse xs)
```

Pour cela, définir op' par op' x y = op y x



Fold (définitions récursives)

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr op e [] = e
foldr op e (x:xs) = op x (foldr op e xs)
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl op e [] = e
foldl op e xs = op (last xs) (foldl op e (init xs))
-- rappel P4
foldr op e xs = foldl op' e (reverse xs)
  where op' x y = op y x
```